LESSON 16

ESP32 SUPER STARTER KIT · 2-HOUR SESSION

Secret Code Lock

Build a keypad door lock with a servo bolt, beeps and warning lights.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- How a **keypad** finds which key you pressed using rows and columns
- How to use the **Keypad** library
- How to build up text in a `String` and compare it
- How to put everything together into a real gadget

SESSION PLAN

0:00	How does a keypad work?
0:10	Install the **Keypad** library and wire the keypad
0:30	Sketch 1: **L16_Keys**
0:45	Add the servo, LEDs and buzzer
0:55	Short break
1:00	Sketch 2: **L16_Lock**
1:25	Challenges
1:50	Crack your neighbour's code!
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ 4×4 membrane keypad
- ☐ Servo motor SG90 + horn
- ☐ Passive buzzer
- ☐ 1 red LED and 1 green LED
- ☐ 2 resistors, 220 Ω (red-red-brown)
- ☐ Jumper wires (8 for the keypad)

1. Sixteen keys, eight wires

The keypad has 16 keys but only **8 wires**. How can that work?

Under the keys are 4 **row** wires going across and 4 **column** wires going down. When you press a key, it connects one row to one column, like where two roads cross.

The ESP32 checks super fast: "*Row 1, are you touching any column? Row 2? Row 3?…*" When it finds a connection at row 2 and column 3, it knows you pressed **6**. This is called **scanning**, and it happens hundreds of times a second.

> **Big idea: rows × columns**
>
> 4 rows × 4 columns = 16 keys, using only 4 + 4 = 8 wires. Computer keyboards use the same trick to read 100 keys!

2. Install the library and wire the keypad

- ☐ **Tools** → **Manage Libraries…**, search **Keypad**, and install *Keypad by Mark Stanley, Alexander Brevig*.
- ☐ You'll need **ESP32Servo** too. You installed it in Lesson 14.

Hold the keypad face up with the ribbon at the bottom. The 8 holes in its plug are, **from left to right**: rows 1–4, then columns 1–4. Push a male jumper wire into each hole.

✓	Keypad hole (left → right)	ESP32 pin
☐	1 (row 1)	**13**
☐	2 (row 2)	**14**
☐	3 (row 3)	**27**
☐	4 (row 4)	**26**
☐	5 (column 1)	**25**
☐	6 (column 2)	**4**
☐	7 (column 3)	**16**
☐	8 (column 4)	**17**

Also connect the buzzer: **+** to pin **33**, **–** to the **–** rail, and ESP32 **GND** to the **–** rail.

> **Use wires of different colours**
>
> It is much easier to check 8 wires if they are in a rainbow order. Then you can see at a glance if two are swapped.

3. Sketch 1: Which key?

File → **New Sketch**, type this in and save as **L16_Keys**. Upload, open the **Serial Monitor**, and press every key.

Type this in → save it as **L16_Keys** — 33 lines

```
// Lesson 16: Read the keypad
// Rows on 13, 14, 27, 26. Columns on 25, 4, 16, 17. Buzzer on 33.

#include <Keypad.h>

const byte ROWS = 4;
const byte COLS = 4;
char keys[ROWS][COLS] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[ROWS] = {13, 14, 27, 26};
byte colPins[COLS] = {25, 4, 16, 17};

Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, ROWS, COLS);

const int BUZZER_PIN = 33;
```

```
void setup() {
  Serial.begin(115200);
  Serial.println("Press some keys!");
}

void loop() {
  char key = keypad.getKey();
  if (key) {
    Serial.print("You pressed: ");
    Serial.println(key);
    tone(BUZZER_PIN, 1500, 50);
  }
}
```

Code	What it means
`#include <Keypad.h>`	Borrow the Keypad library
`char keys[ROWS][COLS]`	A `char` is **one** letter or symbol, in single quotes: `'7'`. This 2D array is a map of the keypad
`byte rowPins[ROWS]`	A `byte` is a small whole number (0–255), perfect for pin numbers
`Keypad keypad = Keypad(…)`	Make a keypad object that knows the map and the pins
`char key = keypad.getKey();`	Which key was just pressed? Gives back `0` (nothing) if none
`if (key)`	True if a key was pressed (anything except 0)

Wrong keys showing?

If pressing **1** shows **D** or *, your 8 wires are in the opposite order. Either turn the plug around, or reverse the numbers in `rowPins` and `colPins`. If one whole row is dead, check that row's wire.

4. Add the lock parts

✓	From	To
☐	ESP32 **5V** (may say **VIN**)	breadboard **+** rail
☐	Servo **brown**	**–** rail
☐	Servo **red**	**+** rail (5 V)
☐	Servo **orange**	ESP32 pin **18**
☐	ESP32 pin **19**	220 Ω resistor → **long leg** of **red** LED
☐	ESP32 pin **23**	220 Ω resistor → **long leg** of **green** LED
☐	short legs of both LEDs	**–** rail

The servo is the door's **bolt**: at 0° it is locked, at 90° it is open.

5. Sketch 2: The lock

File → New Sketch, type this in and save as **L16_Lock**. The secret code is **1234**. Type it, then press **#**.

Type this in → save it as **L16_Lock** — 70 lines

```
// Lesson 16: Secret code lock
// Type the code then #. Press * to start again.

#include <Keypad.h>
#include <ESP32Servo.h>

char keys[4][4] = {
  {'1', '2', '3', 'A'},
  {'4', '5', '6', 'B'},
  {'7', '8', '9', 'C'},
  {'*', '0', '#', 'D'}
};
byte rowPins[4] = {13, 14, 27, 26};
byte colPins[4] = {25, 4, 16, 17};
Keypad keypad = Keypad(makeKeymap(keys), rowPins, colPins, 4, 4);
const int BUZZER_PIN = 33;
const int RED_PIN = 19;
const int GREEN_PIN = 23;
const int SERVO_PIN = 18;

Servo lock;
String secret = "1234";
String typed = "";

void openLock() {
  Serial.println("Correct! Door open.");
  digitalWrite(GREEN_PIN, HIGH);
  lock.write(90);
  delay(5000);
  lock.write(0);
  digitalWrite(GREEN_PIN, LOW);
}

void wrongCode() {
  Serial.println("WRONG CODE!");
  digitalWrite(RED_PIN, HIGH);
  tone(BUZZER_PIN, 200, 800);
  delay(1000);
  digitalWrite(RED_PIN, LOW);
}

void setup() {
  Serial.begin(115200);
  pinMode(RED_PIN, OUTPUT);
```

```
  pinMode(GREEN_PIN, OUTPUT);
  lock.attach(SERVO_PIN, 500, 2400);
  lock.write(0);
}

void loop() {
  char key = keypad.getKey();
  if (!key) {
    return;                          // no key pressed: try again
  }
  tone(BUZZER_PIN, 1500, 50);
  if (key == '*') {
    typed = "";
    Serial.println("Cleared");
  } else if (key == '#') {
    if (typed == secret) {
      openLock();
    } else {
      wrongCode();
    }
    typed = "";
  } else {
    typed = typed + key;
    Serial.println(typed);
  }
}
```

How it works

Code	What it means
`String secret = "1234";`	A `String` holds **text**: lots of characters in double quotes
`String typed = "";`	Starts **empty**. It collects the keys you press
`typed = typed + key;`	Stick the new key on the end: `"12"` + `'3'` → `"123"`
`typed == secret`	Are the two texts exactly the same?
`return;`	Leave `loop()` early. It starts again straight away
`key == '*'`	Single quotes for one character, double quotes for a String

Try it

1. Change the secret code to your own. Can it have letters in it, like `"A7B9"`?
2. Make the door stay open for 10 seconds instead of 5.

6. Challenges

Challenge 1: Secret stars

Anyone watching the Serial Monitor can see your code! Print a `*` for each key instead of the code itself.

Challenge 2: Three strikes

Count wrong tries. After 3 wrong codes in a row, block the keypad for 30 seconds with the red LED flashing. A right code resets the count to 0.

Challenge 3: New code

While the door is open, pressing **A** lets you type a new secret code followed by **#**. *Hint: in* `openLock()`*, wait for keys in a loop instead of* `delay(5000)`*.*

Bonus: Code cracker

Swap computers with a friend (don't peek at their code!). The code is 3 digits. How many tries does it take to crack it? How many 3-digit codes are possible?

7. Quiz

Check what you learned

1. How many wires would a keypad with 5 rows and 6 columns need? How many keys would it have?

   ---

2. What is the difference between `'7'` and `"7"`?

   ---

3. What is in `typed` after you press **4**, **2**, *****, **9**?

Tidy up: unplug the USB cable. Gently pull the jumper wires out of the keypad plug (don't pull the ribbon!).